

Senior Mobile App Developer – Technical Assignment

Experience: 6–8+ Years

Required Experience: Android + Apple/iOS/macOS Application Development

Time Limit: Maximum 3 Hours

Objective

Build a small **Learning Dashboard mobile application**.

We are not evaluating design perfection or the number of screens.

We want to understand:

- Your coding quality
 - Mobile architecture
 - API integration
 - State management
 - Offline handling
 - Debugging approach
 - Production engineering knowledge
-

Assignment

Build the application on **ONE platform of your choice**:

- Android — Kotlin / Jetpack Compose

OR

- iOS/macOS — Swift / SwiftUI

Choose the platform on which you are strongest.

Screen 1 — Login

Create a basic login screen with:

- Email
- Password
- Login button
- Input validation
- Loading state
- Error state

You may mock the login API.

On successful login, navigate to the Course Dashboard.

Screen 2 — Course Dashboard

Fetch and display a list of courses from a mock API/local JSON.

Each course should show:

- Course name
- Instructor name
- Progress %
- Number of lessons
- Continue button

Example:

None

```
[
  {
    "id": 1,
    "title": "Python Programming",
    "instructor": "John Smith",
    "progress": 65,
    "lessons": 20
  },
```

```
{
  "id": 2,
  "title": "Generative AI",
  "instructor": "Sarah Williams",
  "progress": 40,
  "lessons": 16
},
{
  "id": 3,
  "title": "Full Stack Development",
  "instructor": "David Brown",
  "progress": 25,
  "lessons": 28
}
]
```

The screen must handle:

- Loading
- Success
- Empty state
- API failure

Screen 3 — Course Details

Clicking on a course should open a details screen.

Display:

- Course name
- Current progress
- List of lessons

Example:

None

Introduction

✓ Completed

Variables & Data Types

✓ Completed

Functions

○ Pending

OOP

○ Pending

The user should be able to:

Mark a lesson as completed.

Once completed:

- Lesson status should change.
- Course progress should update.

Offline Requirement

After course data has been loaded once:

Turn off the internet.

The user should still be able to see the previously loaded courses.

A simple local database/cache is sufficient.

We are not expecting a complex synchronization engine.

Architecture

Use a clean, maintainable architecture.

For example:

None

UI

↓

```
ViewModel / Presentation
```

```
↓
```

```
Repository
```

```
↓
```

```
API + Local Storage
```

MVVM, Clean Architecture or an equivalent structured approach is acceptable.

We will evaluate whether your architecture is reasonable for a real production application.

Testing

Write at least:

1 meaningful unit test

The test may cover:

- ViewModel
- Repository
- Progress calculation
- Business logic

We prefer **one useful test** over multiple meaningless tests.

README — Maximum 1 Page

Answer the following briefly.

1. Architecture

Why did you choose your architecture?

2. Offline Support

How are you storing and loading offline data?

3. Security

Where would you store authentication tokens in a production application?

4. Scale

If this application had:

1 million users + hundreds of courses

mention 3–5 things you would improve.

5. Second Platform

If you developed Android:

Explain briefly how you would implement the same application on iOS/macOS.

If you developed iOS/macOS:

Explain briefly how you would implement it on Android.

Submission

Please submit:

1. Git Repository

GitHub / GitLab repository containing the complete source code.

2. Build

Android:

- APK

Apple:

- Build/screenshots/simulator recording as applicable.

3. Demo Video

Maximum:

2 minutes

Show:

- Login
- Course list
- Course details
- Lesson completion
- Offline behaviour

4. README

Include the answers requested above.

Important

The assignment has a strict **3-hour limit**.

We are deliberately keeping the scope small.

Do not spend time on:

- Fancy UI
- Animations
- Backend development
- Complex authentication
- Pixel-perfect design
- Extra screens

Prioritize:

Code quality + architecture + working functionality.

AI Tools

You may use any AI Tools. However, during the technical interview you must be able to:

- Explain your code
 - Modify your code
 - Debug your code
 - Explain architectural decisions
-

What We Will Evaluate

Area	Weight
Code quality & structure	20%
Architecture	20%
API & state handling	15%
Offline implementation	15%
Mobile platform knowledge	10%
Error handling	5%
Testing	5%
README / engineering thinking	10%
